Production Cost Minimization and Optimal Energy Flow in Sweden's Electricity Network

Jorben van der Vinne, gusjorva@student.gu.se
Noe Walliser, walliser@chalmers.se
Jakob Rothe, gusrothja@student.gu.se

October 15, 2026

Contents

1 Introduction **2**
1.1 A short Motivation . . . 2
1.2 Background Information: Swedens Energy Grid . . . 2

2 Modeling the Problem **4**
2.1 The Sets and Variables . . . 4
2.2 Objective Function . . . 5
2.3 Constraints . . . 5
2.4 The Model Problem . . . 6

3 Solution **7**
3.1 The Optimal Network Production and Flow . . . 7
3.1.1 Tables . . . 8
3.2 Analyzing the solution . . . 9

4 Summary **10**

A Julia Implementation: Data **11**

B Julia Implementation: Model **13**

Chapter 1

Introduction

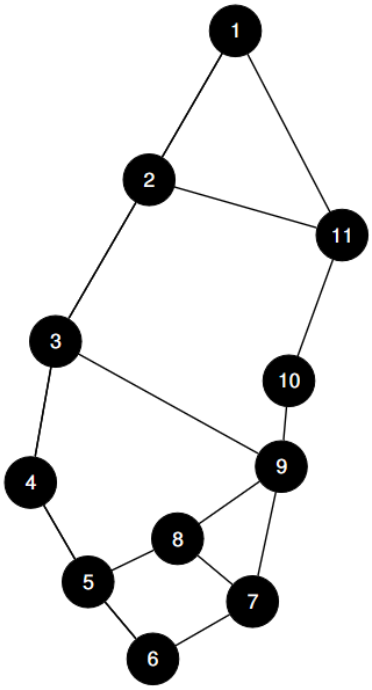
1.1 A short Motivation

In todays times, climate change as well as rising energy costs have become a big problem for more and more citizens. So well done planning of the energy production and distribution is more important than ever before. In this paper we want to study Sweden's Energy Network and find a way to optimize the production and distribution. We will focus on plannable, non sustainable production sources, such as nuclear, hydro, oil etc.

1.2 Background Information: Swedens Energy Grid



Biggest Power Plants in Sweden



Simplified Transmission Grid

In the pictures above you can see a map of Sweden's biggest power plants and a simplified version of the transmission grid. The grid uses an alternating current, operating on 50Hz
In an electrical grid that uses alternating current, power can be divided into two types: active power and reactive power. Active power is the energy that is transferred and used by electrical equipment, while reactive power is energy that is temporarily stored and released by components such as inductors and capacitors. Although both types of power flow through the grid, only active power represents the net energy transferred over one complete AC cycle. When planning electricity generation and transmission, both active and reactive power must be considered. The power flowing between two connected nodes depends on their voltage amplitudes, voltage phase angles, and the electrical properties of the transmission line connecting them.

Chapter 2

Modeling the Problem

2.1 The Sets and Variables

To model the problem we will look at the simplified grid shown earlier. At every node there is at least on generator and/or consumer. Our goal is to find the minimal production cost, while ensuring that costumer demand is satisfied.
We will have a set of Nodes: N=$\{1, 2, 3, ..., 11\}$, the Generators G=$\{1, 2, 3, ..., 9\}$, data about their location, maximum production capacity and production price:

Generator	Location node	Maximum capacity [pu]	Energy production cost [SEK/pu]
G_1	2	0.02	175
G_2	2	0.15	100
G_3	2	0.08	150
G_4	3	0.07	150
G_5	4	0.04	300
G_6	5	0.17	350
G_7	7	0.17	400
G_8	9	0.26	300
G_9	9	0.05	200

Furthermore a set of Consumers C=$\{1, 2, 3, ..., 7\}$, their location and demand:

Generator	Location node	Demand active power [pu]
C_1	1	0.10
C_2	4	0.19
C_3	6	0.11
C_4	8	0.09
C_5	9	0.21
C_6	10	0.05
C_7	11	0.04

Then we also have a set to identify the Connection Lines:
L=$\{(1, 2), (1, 11), (2, 3), (2, 11), (3, 4), (3, 9), (4, 5), (5, 6), (5, 8), (6, 7), (7, 8), (7, 9), (8, 9), (9, 10), (10, 11)\}$
As well as the data, characterizing each line, we need to calculate the voltage amplitude and angle:

Edge (k, ℓ) / Coeff.	$(1,2)$	$(1,11)$	$(2,3)$	$(2,11)$	$(3,4)$	$(3,9)$	$(4,5)$	$(5,6)$	$(5,8)$	$(6,7)$	$(7,8)$	$(7,9)$	$(8,9)$	$(9,10)$	$(10,11)$
$b_{k\ell}$	−20.1	−22.3	−16.8	−17.2	−11.7	−19.4	−10.8	−12.3	−9.2	−13.9	−8.7	−11.3	−7.7	−13.5	−26.7
$g_{k\ell}$	4.12	5.67	2.41	2.78	1.98	3.23	1.59	1.71	1.26	1.11	1.32	2.01	4.41	2.14	5.06

We need to define the variables used in the model, most of which can be taken out of the tables above:
P_i [pu] the active power generated by generator $i \in G$
Q_i [pu] the reactive power generated by generator $i \in G$
P_imax [pu] the maximum of generator $i \in G$
ec_i [SEK/pu] the price of power generation for generator $i \in G$
D_k [pu] the demand of active for every node $k \in N$ (equal to the demand of the consumer at that node, if there is one)
v_k [vu] the voltage amplitude for every node $k \in N$
θ_k [90°] the voltage amplitude for every node $k \in N$
active power flowing from node k to l: $p_{k\ell} = v_k^2 g_{k\ell} - v_k v_\ell g_{k\ell} \cos(\theta_k - \theta_\ell) - v_k v_\ell b_{k\ell} \sin(\theta_k - \theta_\ell)$
reactive power flowing from node k to l: $q_{k\ell} = -v_k^2 b_{k\ell} + v_k v_\ell b_{k\ell} \cos(\theta_k - \theta_\ell) - v_k v_\ell g_{k\ell} \sin(\theta_k - \theta_\ell)$

2.2 Objective Function

The goal to minimize the total cost of produced energy. So we minimize the sum of the production costs resulting from each generator:

$$\min_{\{P,Q,v,\theta\}} \sum_{i=1}^{9} ec_i P_i$$

2.3 Constraints

The main priority for energy production is, that consumer demand is met. So for every node we need to ensure that the total produced acitve power at that node, minus all outgoing active power equals out the demand at that node:

$$(\sum_{i \in G(k)} P_i) - (\sum_{l=1(k,l)\in L}^{11} p_{kl}) - D_k = 0 \quad \forall k \in N$$

Also the reactive power in the grid needs to equal out at every node:

$$(\sum_{i \in G(k)} Q_i) - (\sum_{l=1(k,l)\in L}^{11} q_{kl}) = 0 \; \forall k \in N$$

Lastly all the variables need to stay in their given range:
A generator can only produce as much as its maximum capacity:

$$0 \leq P_i \leq P_imax \; \forall i \in G$$

As mentioned before, the reactive energy can only take a fraction of the maximum capacity:

$$-0,03 P_imax \leq Q_i \leq 0,03 P_imax \; \forall i \in G$$

And voltage amplitude and angle need to stay in a certain range:

$$0,98 \leq v_k \leq 1,02 \; \forall k \in N$$

$$-\pi \leq \theta_k \leq \pi \; \forall k \in N$$

2.4 The Model Problem

Putting all that together, we get a minimization problem, which we will later optimize by computer, using Julia, JuMP and the Ipopt solver:

$$\min_{\{P,Q,v,\theta\}} \sum_{i=1}^{9} ec_i P_i$$

s.t.:

$$(\sum_{i \in G(k)} P_i) - (\sum_{l=1(k,l)\in L}^{11} p_{kl}) - D_k = 0 \quad \forall k \in N$$

$$(\sum_{i \in G(k)} Q_i) - (\sum_{l=1(k,l)\in L}^{11} q_{kl}) = 0 \quad \forall k \in N$$

$$0 \leq P_i \leq P_i max \quad \forall i \in G$$

$$-0,03 P_i max \leq Q_i \leq 0,03 P_i max \quad \forall i \in G$$

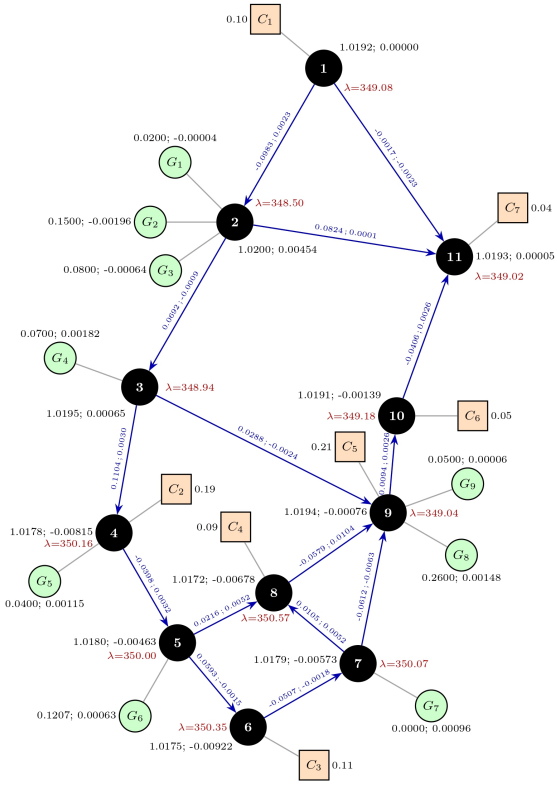
$$0,98 \leq v_k \leq 1,02 \quad \forall k \in N$$

$$-\pi \leq \theta_k \leq \pi \quad \forall k \in N$$

Chapter 3

Solution

3.1 The Optimal Network Production and Flow



Optimal Network Grid

The optimization was solved by using Ipopt, which printed LOCALLY_SOLVED. The minimum total cost of producing the required active power is 183.24 SEK. The total demand is 0.79 pu and the generators together produce 0.7907 pu, so the transmission losses are 0.0007 pu. Generators G1-G5,G8 and G9 run at full capacity and G6 produces the remaining 0.1207 pu of its 0.17 pu. G7 is the most expensive generator and is not used. We see that the cheapest generators are used first. Active power results are in Table 3.1 (P). Reactive power is free, so the generators use it only to keep the voltages within limits. All values are below the limit of ± 3 $The voltage amplitudes lie between 1.0172 vu(node 8) and 1.02 vu(node 2). Higher voltag$
$0.0045 rad(node 2).(Table 3.2) The largest active flow is 0.110 pu online(3,4), followed by 0.098 pu from node 2 to node 1 and line(1,$

3.1.1 Tables

Gen.	Node	Cap. (pu)	P (pu)	Q (pu)	Q limit (pu)	Cost (SEK)
G1	2	0.02	0.0200	−0.00004	±0.0006	3.50
G2	2	0.15	0.1500	−0.00196	±0.0045	15.00
G3	2	0.08	0.0800	−0.00064	±0.0024	12.00
G4	3	0.07	0.0700	0.00182	±0.0021	10.50
G5	4	0.04	0.0400	0.00115	±0.0012	12.00
G6	5	0.17	0.1207	0.00063	±0.0051	42.25
G7	7	0.17	0.0000	0.00096	±0.0051	0.00
G8	9	0.26	0.2600	0.00148	±0.0078	78.00
G9	9	0.05	0.0500	0.00006	±0.0015	10.00
Total			**0.7907**	0.00504		**183.24**

Table 3.1: Optimal active and reactive power for each generator.

Node	v (vu)	θ (rad)
1	1.01915	0 (fixed)
2	1.02000	0.00454
3	1.01949	0.00065
4	1.01777	−0.00815
5	1.01801	−0.00463
6	1.01750	−0.00922
7	1.01791	−0.00573
8	1.01716	−0.00678
9	1.01937	−0.00076
10	1.01908	−0.00139
11	1.01926	0.00005

Table 3.2: Voltage amplitudes and phase angles at the nodes.

The quantities $p_{k\ell}$ and $q_{k\ell}$ are measured at node k's end of the line, and a positive value means flow from k towards ℓ. The loss on a line is the sum of its two ends.

Edge	$p_{k\ell}$	$p_{\ell k}$	active loss	$q_{k\ell}$	$q_{\ell k}$	reactive loss
(1,2)	−0.09829	0.09838	0.00009	0.00228	−0.00184	0.00044
(1,11)	−0.00171	0.00171	0.00000	−0.00228	0.00228	0.00000
(2,3)	0.06922	−0.06918	0.00004	−0.00091	0.00118	0.00027
(2,11)	0.08240	−0.08234	0.00006	0.00010	0.00027	0.00037
(3,4)	0.11040	−0.11023	0.00017	0.00298	−0.00201	0.00098
(3,9)	0.02878	−0.02877	0.00001	−0.00235	0.00239	0.00004
(4,5)	−0.03977	0.03979	0.00002	0.00316	−0.00302	0.00014
(5,6)	0.05934	−0.05930	0.00004	−0.00154	0.00181	0.00027
(5,8)	0.02157	−0.02157	0.00001	0.00519	−0.00514	0.00005
(6,7)	−0.05070	0.05072	0.00001	−0.00181	0.00199	0.00018
(7,8)	0.01051	−0.01051	0.00000	0.00524	−0.00522	0.00001
(7,9)	−0.06122	0.06128	0.00006	−0.00626	0.00658	0.00031
(8,9)	−0.05793	0.05811	0.00019	0.01036	−0.01004	0.00033
(9,10)	0.00938	−0.00938	0.00000	0.00261	−0.00260	0.00001
(10,11)	−0.04062	0.04063	0.00001	0.00260	−0.00254	0.00006

Table 3.3: Active and reactive power flows (pu) on each edge, in the order $k \to \ell$ and $\ell \to k$.

3.2 Analyzing the solution

To know if the problem is convex or not, we have to look at the different functions that are implied in the problem. First of all the objective function in linear in P. However, the feasible region involve some product of variables, and trigonometric function (cos and sin). They don't define a convex set and are not affine. Then the problem is a nonconvex and nonlinear.
When we have our optimal point, we have to know if its local or global. Using the Ipopt solver, we use the interior point method. The interior point method only garantee to converge to a KKT point, satisfying KKT conditions. If the problem is not convex, a KKT point is not garanteed to be a global minimum. However, we can think that this minimum is global for many reasons.
First of all the total active power demand is $0.10 + 0.19 + 0.11 + 0.09 + 0.21 + 0.05 + 0.04 = 0.79$. With our solution, we find that we have a total generation of 0.7909, that number is really close from the previous one. The difference can be explained by the transmission loss in the network. Secondly, the losses and the reactive dispatch are small in magnitude, so we have a "good" local minima, that doesn't push the limit against a contraint.

Chapter 4

Summary

In this project, we looked at how to reduce electricity production costs in a simplified model of Sweden's electricity grid. The goal was to meet the demand while keeping production costs as low as possible and staying within the constraints of the network's components. To solve this, we created an optimization model using Julia, JuMP and the Ipopt solver.

The optimization resulted in a total production cost of 183.244 SEK, while meeting consumer demand.

Overall, it shows how to find a cost-effective way of producing and distributing electricity. All this while still having reasonable complexity and computing time. Implementing this would be a large improvement to unplanned distribution and could help to lower electricity costs for everyone living in Sweden. We need to keep in mind though, that the model is a simplified version of Sweden's electricity grid, further data and a more complex model would be needed to make this work in the real grid. Still the model shows the theory behind how it could be done.

Appendix A

Julia Implementation: Data

```
# Data file for the electric power production and transmission planning
# project (TMA947 / MMG621 - Nonlinear optimization).
#
# Network: 11 nodes, 15 transmission lines, 9 generators, 7 consumers.
# All quantities are already given in normalized (per-unit) values.

nodes = 1:11

# Transmission line data: (from, to, b_kl, g_kl).
# b_kl = b_lk and g_kl = g_lk (the lines are physically symmetric);
# the direction only matters for the power-flow formulas below.
edges = [
    (1, 2, -20.1, 4.12),
    (1, 11, -22.3, 5.67),
    (2, 3, -16.8, 2.41),
    (2, 11, -17.2, 2.78),
    (3, 4, -11.7, 1.98),
    (3, 9, -19.4, 3.23),
    (4, 5, -10.8, 1.59),
    (5, 6, -12.3, 1.71),
    (5, 8, -9.2, 1.26),
    (6, 7, -13.9, 1.11),
    (7, 8, -8.7, 1.32),
    (7, 9, -11.3, 2.01),
    (8, 9, -7.7, 4.41),
    (9, 10, -13.5, 2.14),
    (10, 11, -26.7, 5.06),
]

# Generators: name => (node, maximum active-power capacity [pu], cost [SEK/pu])
generators = Dict(
    "G1" => (node = 2, cap = 0.02, cost = 175.0),
    "G2" => (node = 2, cap = 0.15, cost = 100.0),
    "G3" => (node = 2, cap = 0.08, cost = 150.0),
    "G4" => (node = 3, cap = 0.07, cost = 150.0),
    "G5" => (node = 4, cap = 0.04, cost = 300.0),
    "G6" => (node = 5, cap = 0.17, cost = 350.0),
    "G7" => (node = 7, cap = 0.17, cost = 400.0),
    "G8" => (node = 9, cap = 0.26, cost = 300.0),
    "G9" => (node = 9, cap = 0.05, cost = 200.0),
)
```

```
# Consumers: name => (node, active-power demand [pu])
consumers = Dict(
    "C1" => (node = 1, demand = 0.10),
    "C2" => (node = 4, demand = 0.19),
    "C3" => (node = 6, demand = 0.11),
    "C4" => (node = 8, demand = 0.09),
    "C5" => (node = 9, demand = 0.21),
    "C6" => (node = 10, demand = 0.05),
    "C7" => (node = 11, demand = 0.04),
)

# Bounds
v_lb, v_ub = 0.98, 1.02       # voltage amplitude [vu]
theta_lb, theta_ub = -pi, pi # voltage phase angle [rad]
q_frac = 0.03                 # reactive power = +/- 3% of a generator's capacity
```

Appendix B

Julia Implementation: Model

```
# Nonlinear optimization model for planning production and transmission of
# electric power (TMA947 / MMG621 project).
#
# Minimizes the total cost of active-power production subject to:
#   - the nonlinear AC active/reactive power-flow equations on every line,
#   - active- and reactive-power balance at every node,
#   - generator capacity and voltage/angle operating limits.

using JuMP
import Ipopt

include("power_data.jl")

gen_ids = collect(keys(generators))
cons_ids = collect(keys(consumers))

# Build an adjacency list: for each node k, the list of (neighbour l, b_kl, g_kl)
neighbors = Dict(k => Tuple{Int,Float64,Float64}[] for k in nodes)
for (k, l, b, g) in edges
    push!(neighbors[k], (l, b, g))
    push!(neighbors[l], (k, b, g))
end

the_model = Model(Ipopt.Optimizer)

# ------------------------------ Variables ------------------------------

@variable(the_model, v_lb <= v[nodes] <= v_ub)
@variable(the_model, theta_lb <= theta[nodes] <= theta_ub)

# Reference (slack) bus: the power-flow equations only depend on angle
# DIFFERENCES theta[k] - theta[l], so the model has a redundant rotational
# degree of freedom. Fixing one angle removes this redundancy (a standard
# modelling choice in AC power flow) without changing the achievable
# objective value or any physical quantity of interest.
fix(theta[1], 0.0; force = true)

@variable(the_model, 0 <= P[g = gen_ids] <= generators[g].cap)
@variable(
    the_model,
    -q_frac * generators[g].cap <= Q[g = gen_ids] <= q_frac * generators[g].cap
```

```
)

# ---------------------------------- Objective ----------------------------------

@objective(the_model, Min, sum(generators[g].cost * P[g] for g in gen_ids))

# ------------------- Active / reactive line-flow expressions -------------------

p_flow(k, l, b, g) =
    v[k]^2 * g - v[k] * v[l] * g * cos(theta[k] - theta[l]) -
    v[k] * v[l] * b * sin(theta[k] - theta[l])

q_flow(k, l, b, g) =
    -v[k]^2 * b + v[k] * v[l] * b * cos(theta[k] - theta[l]) -
    v[k] * v[l] * g * sin(theta[k] - theta[l])

# ---------------------------------- Power balance ----------------------------------

@constraint(
    the_model,
    active_balance[k = nodes],
    sum(P[g] for g in gen_ids if generators[g].node == k; init = 0.0) -
    sum(consumers[c].demand for c in cons_ids if consumers[c].node == k; init = 0.0) ==
    sum(p_flow(k, l, b, g) for (l, b, g) in neighbors[k]; init = 0.0)
)

@constraint(
    the_model,
    reactive_balance[k = nodes],
    sum(Q[g] for g in gen_ids if generators[g].node == k; init = 0.0) ==
    sum(q_flow(k, l, b, g) for (l, b, g) in neighbors[k]; init = 0.0)
)

# ---------------------------------- Solve ----------------------------------

optimize!(the_model)

println("Termination status: ", termination_status(the_model))
println("Optimal objective function value (total cost, SEK): ", objective_value(the_mode

println("\nGenerator dispatch:")
for g in gen_ids
    println(
        "  ", g, " (node ", generators[g].node, "): P = ",
        round(value(P[g]); digits = 5), " pu  (cap ", generators[g].cap, ")",
        "   Q = ", round(value(Q[g]); digits = 5), " pu",
    )
end

println("\nNode voltages and phase angles:")
for k in nodes
    println(
        "  Node ", k, ": v = ", round(value(v[k]); digits = 5), " vu   theta = ",
        round(value(theta[k]); digits = 5), " rad",
    )
end
```

```
println("\nActive/reactive power flow on each line (k -> l):")
for (k, l, b, g) in edges
    pkl = value(p_flow(k, l, b, g))
    plk = value(p_flow(l, k, b, g))
    qkl = value(q_flow(k, l, b, g))
    qlk = value(q_flow(l, k, b, g))
    println(
        "  (", k, ",", l, "): p_", k, l, " = ", round(pkl; digits = 5),
        "  p_", l, k, " = ", round(plk; digits = 5),
        "  (loss = ", round(pkl + plk; digits = 5), ")   |  q_", k, l, " = ",
        round(qkl; digits = 5), "  q_", l, k, " = ", round(qlk; digits = 5),
        "  (loss = ", round(qkl + qlk; digits = 5), ")",
    )
end

total_gen = sum(value(P[g]) for g in gen_ids)
total_dem = sum(consumers[c].demand for c in cons_ids)
println(
    "\nTotal active generation: ", round(total_gen; digits = 5),
    " pu, total demand: ", round(total_dem; digits = 5),
    " pu, total transmission losses: ", round(total_gen - total_dem; digits = 5), " pu",
)

# Dual variables (Lagrange multipliers) of the balance constraints give the
# nodal (shadow) prices of active/reactive power — useful for further
# economic analysis, following the pattern shown in the intro example.
println("\nShadow price of active power at each node (dual of active_balance):")
for k in nodes
    println("  Node ", k, ": ", round(dual(active_balance[k]); digits = 3), " SEK/pu")
end
```